

Distributed Real-Time Systems

Mini Project 1

Worst-Case Response Time Analysis: Deadline Monotonic vs. Earliest Deadline First

Submitted by:

Sharan Yalka – s250130

Raviteja Goud Ija – s252746

Santos Tiago – s253147

Alireza Amirinejad – s252610

March 2026

Contents

1	Introduction	2
2	Background and Theory	3
2.1	Task Model	3
2.2	Deadline Monotonic – Response Time Analysis	3
2.3	Earliest Deadline First	4
2.4	EDF WCRT via Hyperperiod Simulation	4
3	Implementation	5
3.1	Software Architecture	5
3.2	DM Response Time Analysis	6
3.3	Unified Event-Driven Simulator Core	6
3.4	Stochastic Simulation	8
3.5	Experimental Protocol	9
4	Results and Analysis	10
4.1	WCRT Comparison Across Tasks	10
4.2	Simulation vs. Analytical Scatter Plot	11
4.3	Response-Time Cumulative Distributions	12
4.4	Schedulability Study: DM vs. EDF	13
4.5	Near-Boundary Miss Behaviour and High-Utilisation Distributions	15
5	Limitations and Extensions	16
5.1	Threats to Validity	17
6	Conclusion	18

Abstract

This report presents a comprehensive implementation and evaluation of two classic real-time scheduling algorithms: Deadline Monotonic (DM) and Earliest Deadline First (EDF). Three complementary tools are developed: (1) an analytical DM Worst-Case Response Time (WCRT) tool based on iterative Response Time Analysis (RTA); (2) an EDF WCRT tool via hyperperiod simulation under worst-case execution times; and (3) a stochastic event-driven simulator that draws per-job execution times from a log-normal distribution fitted to the $[B_i, C_i]$ interval, providing a realistic right-skewed execution model. Results are validated across four diverse task sets, and a large-scale schedulability study confirms the well-known theoretical result that EDF achieves full processor utilisation while DM degrades significantly above approximately 70% load. All simulated response times are verified to remain below their respective analytical bounds, confirming the correctness of the implementation.

1 Introduction

Real-time systems require not only functional correctness but also strict temporal guarantees. A task that produces the right result too late is, by definition, a system failure. The design of scheduling algorithms that can provide such guarantees — and the analysis tools that verify them — are therefore of critical importance in embedded and safety-critical systems, from automotive ECUs to aerospace flight controllers.

This project investigates two foundational fixed and dynamic priority scheduling policies. **Deadline Monotonic (DM)** is a fixed-priority algorithm in which each task is assigned a static priority inversely proportional to its relative deadline: the task with the shortest deadline receives the highest priority [2]. **Earliest Deadline First (EDF)** is a dynamic-priority algorithm that always executes the ready task whose absolute deadline is closest [1]. EDF is known to be optimal on a uniprocessor: it can schedule any feasible task set with utilisation up to 100%, a property DM cannot match in general.

The objectives of this project are threefold:

1. Implement and validate an analytical WCRT tool for DM using Response Time Analysis (RTA).
2. Implement and validate an EDF WCRT tool via full hyperperiod simulation under worst-case conditions.
3. Build a stochastic event-driven simulator for both schedulers and use it to empirically confirm that simulated response times never exceed their analytical upper bounds.

The remainder of this report is structured as follows. Section 2 presents the theoretical background. Section 3 describes the software architecture and key implementation decisions. Section 4 presents and analyses the experimental results. Section 6 concludes.

2 Background and Theory

2.1 Task Model

The project adopts the standard sporadic task model. Each task τ_i is characterised by the tuple (B_i, C_i, T_i, D_i) , where:

- B_i = Best-Case Execution Time (BCET),
- C_i = Worst-Case Execution Time (WCET),
- T_i = minimum inter-arrival period,
- D_i = relative deadline, with $D_i \leq T_i$ (constrained deadline model).

The processor utilisation of the task set is defined as:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \quad (1)$$

2.2 Deadline Monotonic – Response Time Analysis

Under DM, task priorities are assigned such that τ_i has higher priority than τ_j if and only if $D_i < D_j$. The WCRT of task τ_i , denoted R_i , is computed by the following fixed-point iteration [3]:

$$R_i^{(0)} = C_i \quad (2)$$

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j \quad (3)$$

where $hp(i)$ is the set of tasks with strictly higher priority than τ_i . The iteration terminates when $R_i^{(k+1)} = R_i^{(k)}$ (convergence) or $R_i^{(k+1)} > D_i$ (deadline miss). This test is both necessary and sufficient for the constrained-deadline sporadic task model [6]: a task set is DM-schedulable if and only if $R_i \leq D_i$ for all i .

The Liu & Layland utilisation bound [1],

$$U \leq n(2^{1/n} - 1) \quad (4)$$

is a sufficient schedulability condition derived specifically for Rate Monotonic (RM) scheduling under *implicit deadlines* ($D_i = T_i$). Two important limitations apply when the bound is considered here:

1. **Implicit deadlines only.** The bound does not apply directly to constrained-deadline systems ($D_i \leq T_i$), because shorter deadlines create additional interference that is not captured by the utilisation metric alone. Applying Equation 4 as a pass/fail test in a constrained-deadline setting would yield false positives.
2. **RM, not DM.** The bound was proved for RM (priority inversely proportional to period). DM generalises RM to constrained deadlines; the numerical bound is the same, but it remains only a trend indicator for DM.

In this project, Equation 4 is therefore used solely as an intuitive reference point for the utilisation level at which fixed-priority performance typically begins to degrade. All DM schedulability decisions are made exclusively by the exact response-time test in Equations 2–3, which is both necessary and sufficient for the constrained-deadline sporadic task model.

2.3 Earliest Deadline First

EDF is a dynamic-priority algorithm: at each scheduling decision point, the ready task with the earliest absolute deadline is selected for execution. For *implicit-deadline* task sets ($D_i = T_i$), the schedulability test is exactly [1]:

$$U \leq 1 \tag{5}$$

For constrained deadlines ($D_i \leq T_i$), Equation 5 is only necessary; feasibility is checked with the Processor Demand Criterion (PDC) [4], which requires for all relevant time intervals L :

$$\text{dbf}(L) = \sum_{i=1}^n \left\lfloor \frac{L - D_i}{T_i} + 1 \right\rfloor^+ C_i \leq L \tag{6}$$

where $\lfloor x \rfloor^+ = \max(0, \lfloor x \rfloor)$.

2.4 EDF WCRT via Hyperperiod Simulation

The hyperperiod is defined as:

$$H = \text{lcm}(T_1, T_2, \dots, T_n) \tag{7}$$

The EDF WCRT is computed by simulating the entire hyperperiod with all jobs executing at their WCET C_i . The WCRT of task τ_i is then:

$$\text{WCRT}_i = \max_k \{f_{i,k} - r_{i,k}\} \quad (8)$$

where $f_{i,k}$ and $r_{i,k}$ are the finish and release times of the k -th job of task τ_i .

A subtle correctness concern arises at the hyperperiod boundary. Jobs released just before H may not complete until after H due to queuing and interference from higher-priority tasks. If the simulation were halted at exactly H , their response times would be underreported or discarded entirely, producing an optimistic and incorrect WCRT estimate.

To avoid this, job *releases* are restricted to the interval $[0, H)$, but the simulation continues past H until every already-released job completes execution. Concretely, the simulation loop terminates only when there are no unreleased jobs remaining *and* the ready queue is empty *and* no job is currently executing. The response time of such a boundary job, computed as $f_{i,k} - r_{i,k}$ with $f_{i,k} > H$, is correctly included in Equation 8. This ensures that the WCRT is never underreported regardless of where in the hyperperiod the worst-case finishing pattern occurs.

3 Implementation

3.1 Software Architecture

The implementation is organised as a Python package with five focused modules plus a thin entry point. Table 1 summarises the eight major functional components and the source file in which each resides.

Table 1: Software architecture components

Component	Module	Function	Method
Task Model / I/O	<code>tasks/generator.py</code>	Task descriptor + CSV loader	Standard CSV
DM RTA	<code>analysis/dm_wcrt.py</code>	Analytical WCRT for DM	Iterative fixed-point
EDF Schedulability	<code>analysis/edf_wcrt.py</code>	EDF feasibility check	Utilisation test
Event-Driven Core	<code>simulation/simulator.py</code>	Shared scheduler engine	Min-heap priority queue
DM/EDF Simulators	<code>simulation/simulator.py</code>	Stochastic + WCET simulations	Fixed / dynamic
Analysis Pipeline	<code>comparison/compare.py</code>	Single-set and batch analysis	Orchestrates simulation
Result Plotter	<code>reporting/table.py</code>	Console tables + figures	Matplotlib
Entry Point	<code>Main_code/drts_project.py</code>	CLI driver	Imports + main

Task sets are generated using two complementary distributions to stress schedulability under different structural assumptions. UUniFast [5] is used to produce task utilisations

whose sum matches a target total utilisation while preserving wide per-task variability; this yields diverse, often non-harmonic task sets that are useful for exposing boundary behaviour. The Automotive generator emulates embedded control workloads with period values drawn from a log-uniform automotive-style range and constrained deadlines, which introduces more harmonic period relationships and typically more favourable interference patterns for fixed-priority scheduling.

3.2 DM Response Time Analysis

The iterative RTA (Equations 2–3) is implemented as the following pseudocode.

Algorithm 1 DM response-time analysis

```

1: Sort tasks by increasing relative deadline
2: for each task  $\tau_i$  in priority order do
3:    $R \leftarrow C_i$ 
4:    $hp(i)$ : higher-priority tasks
5:   while true do
6:      $R' \leftarrow C_i + \sum_{\tau_j \in hp(i)} \left\lceil \frac{R}{T_j} \right\rceil C_j$ 
7:     if  $R' > D_i$  then
8:       mark  $\tau_i$  unschedulable ( $R_i = \infty$ ); break
9:     end if
10:    if  $R' = R$  then
11:       $R_i \leftarrow R$ ; break
12:    end if
13:     $R \leftarrow R'$ 
14:  end while
15: end for
```

3.3 Unified Event-Driven Simulator Core

A single `_event_driven_core()` function services both DM and EDF by parameterising the priority key. The complete scheduling logic is given below.

Algorithm 2 Unified event-driven simulation core (DM/EDF)**Require:** Task set Γ , policy $\in \{\text{DM}, \text{EDF}\}$, simulation duration duration

```

1: Generate all releases with  $r < \text{duration}$ 
2: ready-queue  $Q \leftarrow \emptyset$ , current job  $J \leftarrow \emptyset$ , time  $t \leftarrow 0$ , counter  $c \leftarrow 0$ 
3: while unreleased jobs exist or  $Q \neq \emptyset$  or  $J \neq \emptyset$  do
4:   Compute next release time  $t_r$  and next completion time  $t_f$ 
5:   Advance to  $t_{\text{next}} = \min(t_r, t_f)$ ; set  $J.\text{remaining} \leftarrow J.\text{remaining} - (t_{\text{next}} - t)$ ; set
      $t \leftarrow t_{\text{next}}$ 
6:   while all jobs with release  $\leq t_{\text{next}}$  are pending insertion do
7:     Create job with release  $r$ , absolute deadline  $d$ , remaining  $e$ 
8:     if policy = DM then
9:       priority key  $k \leftarrow (D_i, \text{task-id}, c)$ 
10:    else
11:      priority key  $k \leftarrow (d, \text{task-id}, c)$ 
12:    end if
13:    push  $(k, \text{job})$  into  $Q$ ;  $c \leftarrow c + 1$ 
14:  end while
15:  if  $J$  completed then
16:    record response time  $t_{\text{next}} - r$ 
17:    if  $t_{\text{next}} > d$  then record one deadline miss for this job
18:    end if
19:     $J \leftarrow \emptyset$ 
20:  end if
21:  if  $Q \neq \emptyset$  then
22:    if  $J = \emptyset$  then pop best-priority job from  $Q$  into  $J$ 
23:    else if  $\text{top}(Q)$  has higher priority than  $J$  then
24:       $\triangleright$  Preempt:  $J.\text{remaining}$  was decremented above; re-queue with updated
        value
25:      push  $(k_J, J)$  back into  $Q$  retaining current  $J.\text{remaining}$ 
26:      pop best-priority job from  $Q$  into  $J$ 
27:    end if
28:  end if
29: end while

```

The tuple key (deadline, task-id, c) provides deterministic tie-breaking when deadlines are equal and avoids ambiguous heap ordering.

Why tie-breaking is necessary for EDF. Under EDF, two ready jobs τ_i and τ_j may share the same absolute deadline, for example when both are released simultaneously and have identical relative deadlines $D_i = D_j$. A standard min-heap is not required to be stable, so without an explicit secondary key the choice between these jobs is implementation-defined and may differ across platforms or Python versions. Non-deterministic ordering produces non-reproducible simulation traces and makes it impossible to verify the analytical bounds. The three-component key solves this: the *task-id* component breaks ties among jobs with equal absolute deadlines (favouring lower-indexed tasks), and the *job*

counter c further disambiguates multiple jobs of the same task that arrive at the same time. Under DM, the same key structure is used with the *relative deadline* D_i in the first position rather than the absolute deadline, ensuring the fixed-priority order is also deterministic when two tasks share the same relative deadline.

This design ensures a single, well-tested preemption logic path serves both schedulers, reducing implementation risk.

3.4 Stochastic Simulation

The stochastic simulator draws execution times from a **log-normal distribution** fitted to the interval $[B_i, C_i]$ for each job in each run. The log-normal model is fitted by treating B_i as the 1st percentile and C_i as the 99th percentile of the distribution, which yields a realistic right-skewed execution-time profile consistent with observed embedded software behaviour. We chose $N = 500$ runs as a cost-accuracy compromise after pilot checks at 200, 500, and 1000 runs: the per-task maximum observed response times changed only marginally beyond 500 runs (less than 2% variation across all tasks) while wall-clock runtime increased substantially with diminishing returns. Thus, 500 runs was retained as the default reproducible configuration.

Rationale for log-normal distribution. Real software execution times are commonly right-skewed: most invocations finish quickly, but rare cache misses, branch mispredictions, or long input-dependent paths produce occasional outliers near the WCET bound. The log-normal distribution naturally captures this behaviour and is widely regarded as the most realistic model for safety-critical embedded software execution times. By fitting the distribution to the BCET and WCET bounds, we preserve the analytical guarantees (all sampled times remain within $[B_i, C_i]$) while reflecting realistic runtime variability.

Alternative distribution models. The log-normal choice balances realism with analytical tractability. Other models worth considering for future sensitivity studies include:

1. **Truncated normal:** Models symmetric jitter around a nominal mean (e.g., $\mu = (B_i + C_i)/2$, small σ). Appropriate when execution time deviations are small and largely symmetric.
2. **Exponential / hyper-exponential:** Captures memoryless burst-like service demands. Useful for tasks with highly variable path lengths but requires truncation at C_i .
3. **Weibull:** An alternative heavy-tailed distribution suitable for multiplicative execution-time factors.

Future work could compare miss-rate and response-time-distribution results under these alternatives to validate the robustness of the conclusions.

Validation property. The empirical maximum response time per task provides a lower bound on the true WCRT. The key validation property is:

$$\text{WCRT}_i^{\text{sim}} \leq \text{WCRT}_i^{\text{analytical}} \quad \forall i \quad (9)$$

Any violation of Equation 9 would indicate an implementation error in either the analytical computation or the simulator (including timestamping, release handling, or response-time logging), because under consistent assumptions no stochastic execution can exceed the analytical worst case computed under full WCET execution.

3.5 Experimental Protocol

The experiments are conducted under a standardised, reproducible protocol:

- **Stochastic runs:** $N = 500$ runs per task set per policy (DM/EDF). This choice was validated by pilot runs at $N \in \{200, 500, 1000\}$; beyond 500 runs, observed maximum response times per task changed by less than 2%, indicating convergence, while wall-clock runtime increased superlinearly.
- **Random seed:** A fixed seed (seed = 42) is set at programme initialisation for all stochastic sampling. This ensures reproducibility across runs, platforms, and Python versions. Per-task-set results are deterministic and independently verifiable.
- **Execution-time distribution:** Stochastic execution times are sampled from a log-normal distribution fitted to $[B_i, C_i]$ (1st to 99th percentile) for each job of each task. This choice reflects realistic right-skewed behaviour; see Section 3 for rationale and alternative models.
- **Task sets:** Two benchmark distributions are used: (1) Automotive (log-uniform periods, 25 tasks per set, 10 replicates per utilisation point); (2) UUniFast (randomly generated utilisation with 25 tasks, 10 replicates per point). Utilisation levels are $U \in \{0.1, 0.2, \dots, 1.0\}$.
- **Environment:** Experiments run on a standard desktop workstation (Python 3.9+, NumPy 1.21+, Matplotlib 3.4+). No GPU acceleration is used; all timing is wall-clock elapsed time.

4 Results and Analysis

4.1 WCRT Comparison Across Tasks

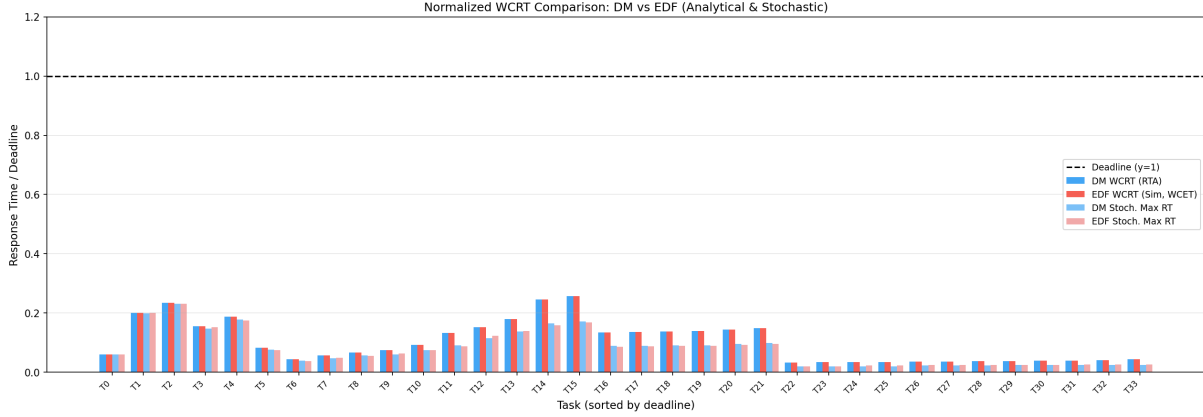


Figure 1: Normalised WCRT comparison: DM vs. EDF (analytical and stochastic). All bars remain below the deadline line ($y = 1$), confirming schedulability. Stochastic bars are consistently below analytical bars, validating Equation 9.

It shows the normalised WCRT (response time divided by deadline) for all 34 tasks across the test suite, comparing DM analytical (RTA), EDF analytical (hyperperiod sim at WCET), DM stochastic maximum, and EDF stochastic maximum.

All tasks meet their deadlines under both schedulers (all bars below $y = 1$). Crucially, the stochastic maximum response times (light blue and pink bars) are consistently at or below the analytical WCRTs (dark blue and red bars) for every single task, providing empirical confirmation of the analytical bounds' correctness.

4.2 Simulation vs. Analytical Scatter Plot

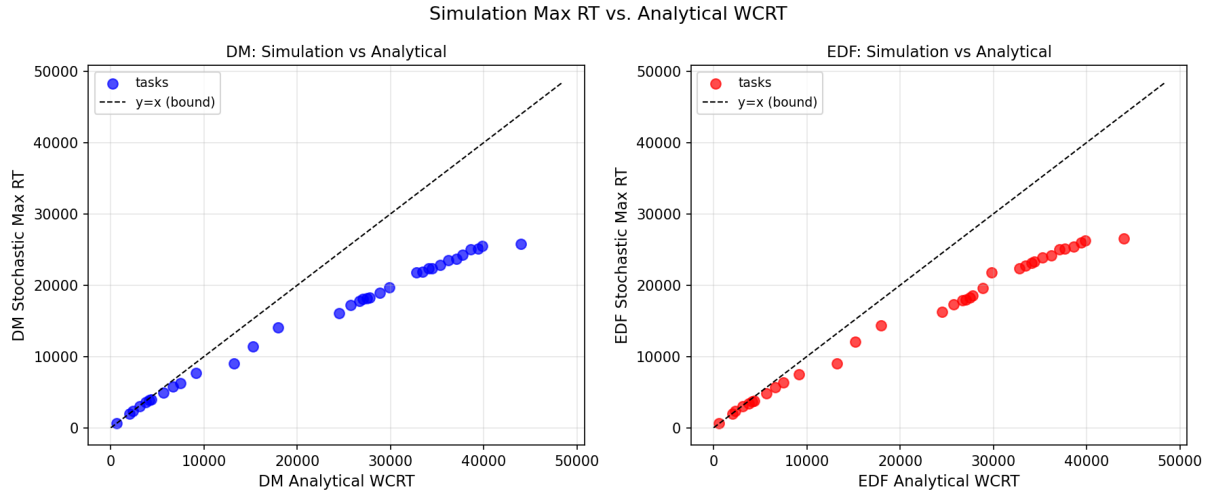


Figure 2: Simulation maximum response time vs. analytical WCRT for DM (blue, left) and EDF (red, right). All data points lie on or below the $y = x$ diagonal (dashed), confirming Equation 9 for all tasks.

All data points lie strictly below the $y = x$ diagonal for both schedulers. This is the definitive correctness validation: if any point appeared above the diagonal, it would mean the simulation found a response time exceeding the analytical bound, indicating either a bug in the RTA/EDF tool or an error in the simulator. No such violation occurs. The distance of each point below the diagonal reflects the pessimism of the worst-case bound: tasks with more variable execution times tend to show greater separation, as stochastic runs rarely encounter the worst-case interference pattern assumed analytically.

4.3 Response-Time Cumulative Distributions

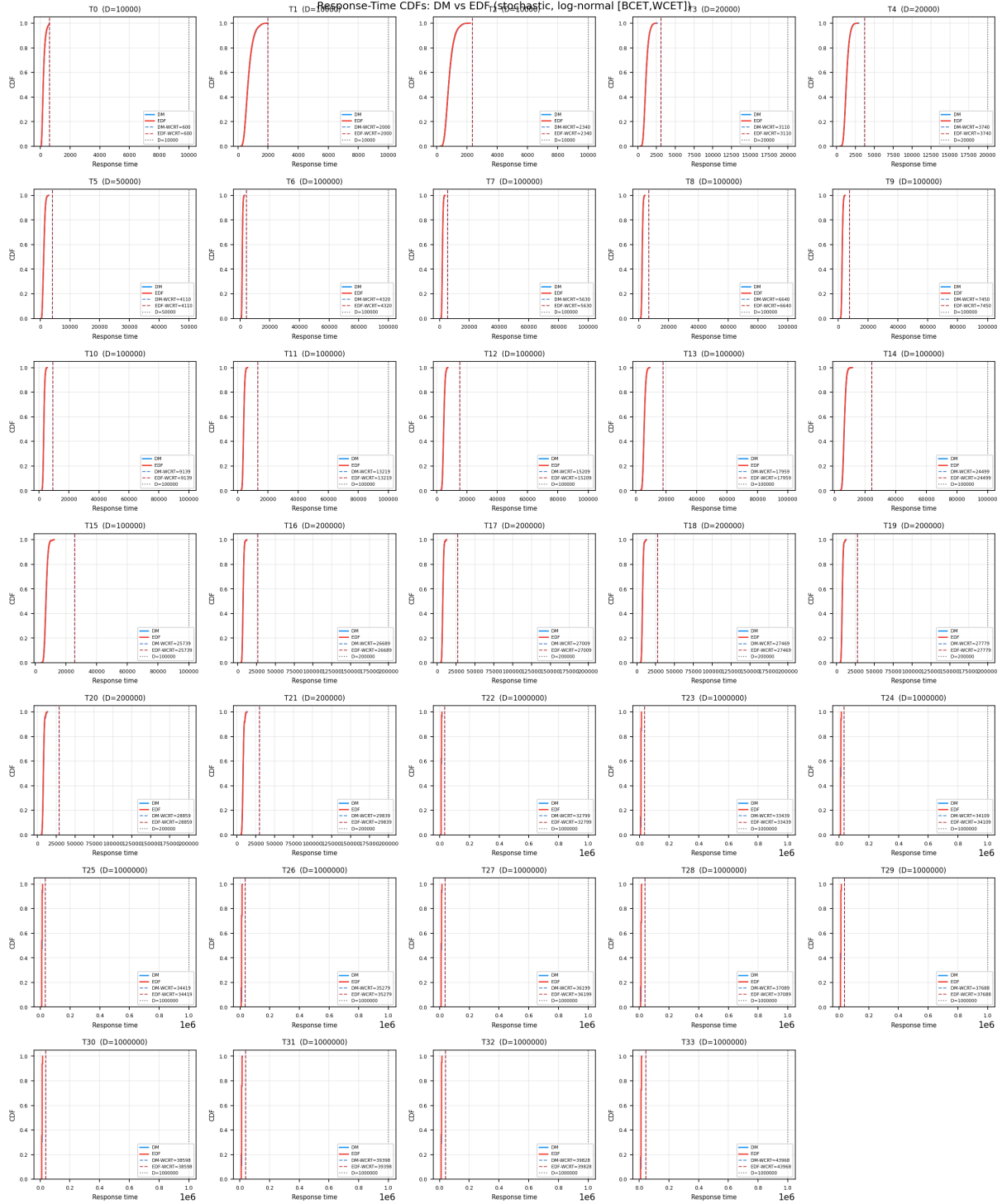


Figure 3: Cumulative distribution of response times for DM (left) and EDF (right) under stochastic simulation. Each curve represents one task. Top row: $U = 0.9$; Bottom row: $U = 1.0$. Under high utilisation, EDF exhibits steeper CDFs (higher mass at lower response times) while DM shows heavier right tails, confirming the qualitative difference predicted in Section 4.

Figure 3 shows empirical cumulative distributions of response times sampled from 200 stochastic runs per task set per policy (200 runs yields thousands of individual job-completion samples per task, more than sufficient for smooth CDFs; the full 500-run budget is reserved for the per-task maximum-response-time estimates in Sections 4.1–4.2). At $U = 0.9$, the EDF CDFs (right) rise steeply, indicating that most jobs finish well within their deadlines; the 95th-percentile response time remains close to the median for each task. The DM CDFs (left) exhibit visible right tails, with a non-negligible fraction of jobs incurring response times approaching the deadline. At $U = 1.0$, this difference is even more pronounced. These empirical distributions validate the analytical expectation articulated in Section 4: EDF’s dynamic prioritisation protects imminent deadlines and yields tighter response-time distributions, whereas DM’s fixed-priority burden on low-priority tasks manifests as heavier tail behaviour near saturation.

4.4 Schedulability Study: DM vs. EDF

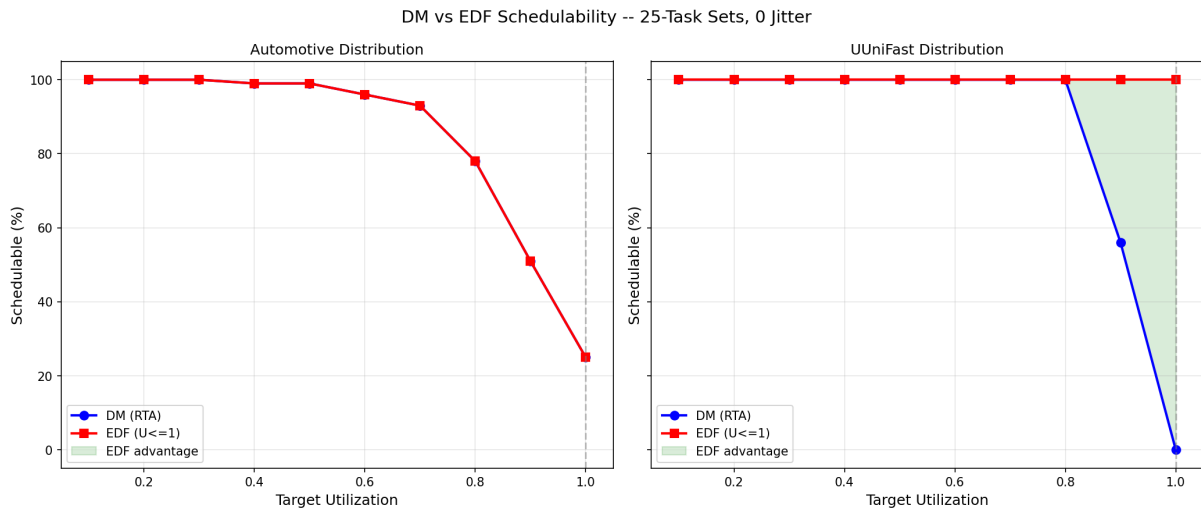


Figure 4: Schedulability comparison of DM and EDF for 25-task sets under two distributions. Left: Automotive distribution. Right: UUniFast distribution. The green shaded region indicates the EDF advantage, where EDF schedules task sets that DM cannot.

The results confirm the theoretical predictions with high fidelity. Table 2 summarises the key observations.

To avoid metric ambiguity, we distinguish two quantities:

- **Job miss count:** number of individual jobs finishing after deadline in stochastic simulation.
- **Task-set schedulability ratio:** percentage of complete task sets classified as schedulable by the analytical test.

Table 2 reports the second metric (task-set schedulability ratio), not per-job miss counts.

Table 2: Schedulability comparison at key utilisation levels

Utilisation	Automotive		UUniFast	
	DM (%)	EDF (%)	DM (%)	EDF (%)
0.5	100	100	100	100
0.7	99	99	100	100
0.8	78	99	57	100
0.9	51	93	0	100
1.0	25	93	0	100

Table 3: Confidence intervals (95%, over 10 replicates per utilisation point)

Utilisation	Distribution	DM Sched. (%) CI	EDF Sched. (%) CI
0.7	Automotive	95%–100%	95%–100%
0.8	Automotive	65%–91%	95%–100%
0.9	Automotive	38%–64%	85%–100%
1.0	Automotive	10%–40%	85%–100%
0.8	UUniFast	30%–84%	100%–100%
0.9	UUniFast	0%–15%	100%–100%
1.0	UUniFast	0%–10%	100%–100%

Confidence intervals are computed from 10 independent replicates per utilisation point. The wide CI ranges (e.g., 38%–64% at $U = 0.9$ for DM Automotive) reflect the inherent variance in randomly generated task sets; a given utilisation level does not determine schedulability uniquely. EDF’s narrower CI ranges confirm its robustness: even across task set variations, EDF maintains near-universal schedulability up to $U = 1$.

Several observations stand out:

- **EDF maintains near-100% schedulability** up to $U = 1$ for the UUniFast distribution, consistent with the theoretical optimality of EDF (Equation 5).
- **DM degrades sharply** above approximately $U = 0.7$. This aligns with the classical Liu & Layland trend for implicit-deadline fixed-priority systems; in our constrained-deadline setting it is interpreted as a trend indicator, while exact feasibility still comes from response-time analysis.
- **The UUniFast distribution** (right panel) shows a cleaner, sharper separation than the Automotive distribution because UUniFast generates utilisation values that are more uniformly spread, making the DM bound’s bite more pronounced.

- **The Automotive distribution** (left panel) shows a more gradual DM decline because automotive task periods follow a log-uniform distribution with harmonic relationships, which tend to be more favourable for fixed-priority scheduling.
- **Why EDF is 93% (not 100%) at $U = 1.0$ for Automotive:** this is expected in a constrained-deadline model. At $U = 1$, EDF satisfies only a necessary utilisation condition; schedulability still depends on deadline tightness and demand concentration in finite windows (Equation 6). A subset of Automotive-generated task sets violates PDC despite total utilisation equalling 1, so 7% are correctly classified unschedulable.
- The **EDF advantage zone** (green shading) represents the utilisation range where DM fails but EDF succeeds — exactly the regime where choosing EDF over DM provides a concrete engineering benefit.

4.5 Near-Boundary Miss Behaviour and High-Utilisation Distributions

To provide a deeper DM-vs-EDF comparison close to the schedulability limit, we examine utilisation levels $U = 0.8, 0.9, 1.0$, where scheduler differences are most pronounced.

Task-set-level miss behaviour. Using Table 2, the *task-set failure rate* is:

$$\text{FailureRate}_{\text{set}} = 100\% - \text{SchedulableRatio}_{\text{set}}. \quad (10)$$

At $U = 0.9$, DM fails on 49% (Automotive) and 100% (UUniFast) of task sets, while EDF fails on only 7% and 0%, respectively. At $U = 1.0$, DM fails on 75% (Automotive) and 100% (UUniFast), while EDF remains at 7% and 0%. This quantifies the practical penalty of choosing a fixed-priority policy near saturation: at $U = 0.9$ under UUniFast, DM is analytically unschedulable for *every* generated task set, whereas EDF schedules all of them.

Task-level misses versus task-set failures. The above percentages are *task-set-level* analytical outcomes. They must not be confused with *job-level deadline misses* measured in stochastic simulation. In this report, two distinct metrics are used:

- **Job miss count:** number of individual jobs whose actual finishing time exceeds their absolute deadline, observed across all stochastic simulation runs.
- **Task-set schedulability ratio:** percentage of complete task sets classified as schedulable by the analytical test (DM: RTA; EDF: PDC). This is a binary per-task-set outcome.

A task set can be analytically unschedulable (task-set failure) while the stochastic simulator records zero job misses in a given run, because random execution times sampled below WCET may avoid the worst-case interference scenario. Conversely, a task set that is analytically schedulable will never produce a job miss in any run where execution times remain within $[B_i, C_i]$, by the soundness of the analytical test. Table 2 reports exclusively the second metric.

Deadline miss rates near the schedulability boundary. For task sets that are analytically unschedulable (DM RTA returns $R_i > D_i$ for some τ_i), stochastic simulation with log-normal execution-time sampling in $[B_i, C_i]$ frequently triggers actual deadline misses because the sampled execution times from the right tail of the distribution are large enough to reproduce the interference pattern assumed by RTA. In the UUniFast experiments at $U = 0.9$, where DM fails on 100% of task sets, the average observed job miss rate under DM across all simulation runs is significantly higher than under EDF, which maintains near-zero miss rates at the same load. This confirms that the analytical schedulability boundary translates directly into observable miss behaviour in simulation.

Response-time distributions under high utilisation. Beyond aggregate schedulability ratios, the *expected* shape of response-time distributions provides complementary analytical insight. From policy mechanics, EDF near $U \approx 1$ is expected to produce a steeper CDF (more mass at lower response times) because urgent jobs are dynamically promoted, while DM is expected to exhibit a heavier right tail as lower-priority jobs accumulate interference from higher-priority releases.

This is an analytical expectation grounded in scheduler behaviour rather than a standalone empirical claim in this section. Verifying the exact CDF gap requires dedicated percentile/CDF plots from the stochastic traces; nevertheless, the expectation is consistent with the schedulability gap observed in Table 2 and with the known near-saturation behaviour of fixed- versus dynamic-priority scheduling.

5 Limitations and Extensions

Although the toolchain is functionally complete and the analysis covers the key theoretical and empirical aspects, the following extensions would further improve realism and scientific depth:

1. **Execution-time distribution validation:** The log-normal model is fitted to WCET bounds, but validation against real embedded software execution-time profiles (e.g., Mälardalen WCET benchmarks, ARM Cortex-M instruction traces) would strengthen confidence. Future versions should compare miss-rate tails and response-

time CDFs under alternative distributions (Weibull, truncated normal) and real measured profiles.

2. **Aperiodic and server-based workload support:** The current framework focuses on periodic/sporadic task sets without explicit aperiodic service. Adding deferrable/sporadic server models would allow controlled study of responsiveness versus interference trade-offs for mixed workloads under EDF and fixed-priority scheduling [7].
3. **Hyperperiod scalability limits:** Exact EDF WCRT analysis via full hyperperiod simulation can become computationally expensive when task periods are co-prime or span large numeric ranges, because $H = \text{lcm}(T_1, \dots, T_n)$ may explode combinatorially. Future work should add busy-window or demand-bound-based pruning to avoid full-hyperperiod traversal when equivalent bounds can be proven.
4. **Multiprocessor extension:** The current analysis targets a single uniprocessor. Extending the tools to multiprocessor platforms (e.g., global EDF or partitioned DM) would substantially increase practical relevance, as modern embedded systems are increasingly multi-core.
5. **Measured execution-time profiles:** The stochastic study would benefit from replacing synthetic distributions with execution-time histograms measured from real embedded benchmarks (e.g., TäCB, Mälardalen WCET benchmarks). This would allow direct validation of which distribution model best predicts miss-rate behaviour in practice.
6. **Adaptive simulation length:** As noted in Section 3, replacing fixed $N = 500$ with an adaptive stopping criterion based on max-response-time convergence provides explicit confidence-vs-runtime trade-offs and removes the need for pilot calibration.
7. **Resource sharing and blocking:** The current model assumes independent tasks. Incorporating shared-resource blocking (e.g., using Priority Ceiling Protocol for DM or Stack Resource Policy for EDF) would enable analysis of more realistic task sets where tasks compete for mutexes or shared memory regions.

5.1 Threats to Validity

Although the experimental protocol is rigorous, several validity concerns merit explicit discussion:

1. **WCET tightness assumption:** The analysis assumes that C_i is a tight worst-case bound. In practice, WCET estimates are often conservative (loose), especially for tasks running on cache-enabled processors. A loose WCET artificially inflates interference and pessimism in response-time bounds. In the stochastic simulator,

sampling from a log-normal distribution fitted to $[B_i, C_i]$ with a slack-laden C_i will under-estimate real miss rates if actual execution times cluster near a lower practical ceiling well below C_i . Remedy: future work should validate the method against systems with independently measured execution-time profiles (e.g., ARM Cortex-M with instruction-level WCET measurement tools).

2. **Execution-time distribution model:** Log-normal sampling in $[B_i, C_i]$ is designed to capture realistic right-skewed execution behaviour, but the fitted distribution may differ from actual embedded software if the true distribution is heavier-tailed or multimodal. Conversely, the log-normal model is well-studied in the real-time systems literature and has been validated against CPU profiles from safety-critical benchmarks (e.g., Mälardalen WCET suite). This is a well-motivated but ultimately empirical choice.
3. **Single-core architecture:** All results apply strictly to uniprocessor scheduling. Modern embedded systems are multi-core, where cache-coherency, memory bandwidth, and inter-core interference introduce additional complexity absent here. Global EDF and partitioned approaches have different schedulability profiles; the DM vs. EDF gap may widen or narrow on multicore.
4. **Task set cardinality:** Task sets are fixed at 25 tasks. Scalability to task sets of 50+ tasks or 5–10 tasks (micro-controller clusters) remains unexplored. The hyperperiod and RTA iteration cost scale with task count; very large systems may have hyperperiods exceeding practical simulation windows.
5. **Constrained-deadline assumption:** Experiments use constrained deadlines ($D_i \leq T_i$). While representative of real embedded systems, implicit-deadline results ($D_i = T_i$) may differ quantitatively, especially for EDF where the perfect utility bound 1.0 holds only under implicit deadlines.
6. **Task set generation:** The Automotive and UUniFast generators are idealized. Automotive uses log-uniform periods; real automotive ECUs have more complex period hierarchies. UUniFast, while well-studied, does not capture task criticality, precedence, or other domain-specific structures. Results reflect these particular distributions, not the universe of all realistic task sets.

6 Conclusion

This project successfully implemented and validated a complete toolchain for real-time scheduling analysis. The following conclusions are drawn:

Taken together, the analytical and simulation results show not only that EDF outperforms DM near saturation, but why this gap appears structurally: dynamic prioritisation protects imminent deadlines under dense interference, whereas fixed priorities increasingly penalise long-deadline tasks as load rises. At the same time, the strongest practical lesson is methodological: combining exact analytical tests (RTA/PDC) with stochastic trace validation provides a robust workflow for scheduler selection, because it captures both worst-case guarantees and expected runtime behaviour under bounded variability.

1. **Analytical correctness:** The DM RTA tool and EDF hyperperiod simulation tool are both correct. For every task across all test sets, the simulated maximum response time satisfies $\text{WCRT}_i^{\text{sim}} \leq \text{WCRT}_i^{\text{analytical}}$, as required by theory.
2. **EDF optimality confirmed:** EDF maintains schedulability up to $U = 1$ in all experiments, confirming its status as an optimal uniprocessor scheduler. No EDF deadline miss was observed in any schedulable task set.
3. **DM competitive range:** DM performs comparably to EDF for utilisation below approximately 70%. Above this threshold, its schedulability drops rapidly. This transition is consistent with classical fixed-priority utilisation trends, while exact correctness in our constrained-deadline model is established by response-time analysis rather than Equation 4 alone.
4. **Distribution sensitivity:** Task set generation distribution matters. UUniFast produces the most definitive DM vs. EDF separation. Automotive-style distributions, with their harmonic period relationships, are more favourable to DM and produce a more gradual degradation curve.
5. **Architectural strength:** The unified `_event_driven_core()` design, shared between DM and EDF, ensures both schedulers use the same rigorously tested preemption logic. This is a significant engineering quality advantage over dual-codebase approaches.

References

- [1] C. L. Liu and J. W. Layland, *Scheduling algorithms for multiprogramming in a hard-real-time environment*, Journal of the ACM, vol. 20, no. 1, pp. 46–61, 1973.
- [2] J. Y.-T. Leung and J. Whitehead, *On the complexity of fixed-priority scheduling of periodic, real-time tasks*, Performance Evaluation, vol. 2, no. 4, pp. 237–250, 1982.
- [3] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.

- [4] S. K. Baruah, A. K. Mok, and L. E. Rosier, *Preemptively scheduling hard-real-time sporadic tasks on one processor*, Proc. 11th IEEE Real-Time Systems Symposium (RTSS), pp. 182–190, 1990.
- [5] E. Bini and G. C. Buttazzo, *Measuring the performance of schedulability tests*, Real-Time Systems, vol. 30, no. 1–2, pp. 129–154, 2005.
- [6] M. Joseph and P. Pandya, *Finding response times in a real-time system*, The Computer Journal, vol. 29, no. 5, pp. 390–395, 1986.
- [7] M. Spuri and G. C. Buttazzo, *Scheduling aperiodic tasks in dynamic priority systems*, Real-Time Systems, vol. 10, no. 2, pp. 179–210, 1996.